

FrameLabs

Collaborative Engineering Diagram Workspace Platform

Document Type	Product Requirement Document
Version	1.0
Program	DevClub Product Builders Program
Duration	8 Weeks
Team Structure	4 – 5 Engineers
Status	Draft

Table of Contents

1 Product Overview

2 Product Vision

3 Similar Product Inspirations

- [Eraser.io](#)
- [Excalidraw](#)
- [Mermaid](#)
- [PlantUML](#)
- [dbdiagram.io](#)

4 Target Users

- [Software Engineers](#)
- [Engineering Teams](#)
- [Technical Leads & Architects](#)

5 Expected Product Outcome

6 Core Product Requirements

- [6.1 Diagram Workspace System](#)
- [6.2 Dual Editing System](#)
- [6.3 Database Design Workspace](#)
- [6.4 System Architecture Workspace](#)
- [6.5 Sequence Diagram Workspace](#)
- [6.6 Class Diagram Workspace](#)
- [6.7 Intelligent Component System](#)
- [6.8 Smart Connection System](#)
- [6.9 Collaboration System](#)
- [6.10 Version History System](#)
- [6.11 Organisation System](#)
- [6.12 Export & Sharing System](#)

7 Product Scale Requirements

8 Product Constraints

9 Technical Expectations

10 Success Metrics

11 Final Deliverables

1. Product Overview

Product Summary

FrameLabs is a collaborative engineering diagramming platform that enables developers and technical teams to design, document, and communicate software systems visually.

The platform provides specialised diagram workspaces where engineers can create system architectures, database designs, application flows, and software structures using both visual interactions and syntax-based definitions.

Positioning Constraint

FrameLabs should NOT behave like a generic whiteboard application.

The objective is a dedicated engineering workspace where diagrams represent actual technical systems, relationships, and architectural decisions.

2. Product Vision

Before software systems are implemented, they are designed. Engineering teams regularly create diagrams to understand system architecture, database relationships, application flows, service communication, and code structure.

However, engineering diagrams are often scattered across different tools, outdated images, or manually maintained files.

The vision of FrameLabs is to create a single workspace where technical diagrams become living engineering documents.

The platform should allow engineers to:

- Design systems visually
- Represent technical relationships accurately
- Collaborate with teammates in real time
- Maintain architecture history over time
- Document evolving software systems

Vision Statement

The goal is not drawing.

The goal is helping engineers think, design, and communicate better.

3. Similar Product Inspirations

Teams should study existing engineering and diagramming platforms. The goal is understanding workflows and product decisions — not replicating interfaces.

Eraser.io

Focus Areas	Engineering-focused diagrams, Architecture documentation
Key Learnings	Diagram-as-code workflows, Developer experience design

Excalidraw

Focus Areas	Fast diagram creation, Simple interaction design
Key Learnings	Collaboration UX, Low-friction diagramming

Mermaid

Focus Areas	Syntax-based diagram creation, Text-to-diagram workflows
Key Learnings	Documentation integration, Developer-first authoring

PlantUML

Focus Areas	UML generation, Structured diagram definitions
Key Learnings	Software modelling workflows, Schema-driven diagrams

dbdiagram.io

Focus Areas	Database schema modelling, Relationship representation
Key Learnings	Code-generated diagrams, ERD design patterns

4. Target Users

Software Engineers

Developers designing and documenting technical systems.

Expected needs:

- Create architecture and system diagrams
- Represent code structure and component relationships
- Document architectural decisions
- Share designs with teammates

Engineering Teams

Groups collaborating on system design and documentation.

Expected needs:

- Shared diagram workspaces
- Real-time architecture discussions
- Version tracking across contributors
- Concurrent collaborative editing

Technical Leads & Architects

Users responsible for high-level system planning and communication.

Expected needs:

- Large-scale system representation
- Maintaining architecture evolution over time
- Communicating technical decisions to teams

5. Expected Product Outcome

The final product should provide engineers with a complete technical diagramming environment.

Users should be able to:

- Create different types of engineering diagrams in specialised workspaces
- Generate diagrams through syntax-based definitions
- Edit diagrams visually through drag-and-drop interactions
- Collaborate with teammates simultaneously
- Manage and browse diagram versions
- Export diagrams as images, PDFs, and documentation formats
- Maintain system designs over extended time periods

Positioning Statement

FrameLabs should feel like an engineering tool — not a drawing tool.

6. Core Product Requirements

6.1 Diagram Workspace System

Users should create specialised workspaces based on the type of system they want to design. FrameLabs must not provide one generic canvas for every purpose.

Supported diagram workspace types:

Database Diagram	Schema definition, tables, relationships, constraints
System Architecture	Services, APIs, infrastructure, data flow, dependencies
Sequence Diagram	Component interactions, request-response flows, timelines
Class Diagram	OOP structure, inheritance, associations, interfaces
Flow Diagram	Process logic, conditional paths, decision trees

Each workspace must provide relevant components, diagram-specific tools, correct relationship types, and an appropriate editing experience for its domain.

6.2 Dual Editing System

Different engineers prefer different workflows. FrameLabs should support both visual and syntax-based diagram creation without forcing one approach.

Visual Editing

- Create and position components via drag-and-drop
- Draw connections and define relationships interactively
- Edit properties through visual panels

Syntax-Based Editing

- Text definitions generate corresponding diagrams
- Diagram changes propagate back to the structure definition
- Users can switch between visual and syntax modes freely

Example: Dual Editing in Practice

An engineer defines a database schema in text (e.g. DBML or SQL DDL).
FrameLabs automatically generates the visual ERD representation.
Editing either the visual diagram or the text keeps both in sync.

6.3 Database Design Workspace

A specialised environment for database modelling. The workspace must understand database semantics — not treat everything as generic shapes.

Schema Definition capabilities:

- Define tables, columns, and data types
- Set primary keys, foreign keys, and constraints
- Add indices and default values

Relationship Management:

- One-to-one relationships
- One-to-many relationships
- Many-to-many relationships with junction table support

6.4 System Architecture Workspace

A specialised environment for designing software architectures at the macro level.

Supported components:

- Frontend applications and client interfaces
- Backend services and microservices
- APIs and gateways
- Databases and data stores
- Caches and message queues
- External third-party systems
- Infrastructure and cloud components

Modelling capabilities:

- Communication flow between services
- System dependencies and boundaries
- Data movement and transformation paths

6.5 Sequence Diagram Workspace

A dedicated workspace for modelling interactions between systems and actors over time.

Define:

- Users and actors
- Services and components
- Communication sequences and ordering
- Request-response pairs and async flows

Example Sequence Scenarios

Authentication flows (login, token refresh, logout)
Payment processing pipelines
API request-response workflows

6.6 Class Diagram Workspace

A workspace for representing object-oriented software structure.

Create:

- Classes and abstract classes
- Interfaces and traits
- Attributes with types and visibility
- Methods with signatures

Represent relationships:

- Inheritance and implementation
- Associations and aggregations
- Dependencies and compositions

6.7 Intelligent Component System

Diagram objects should carry meaning beyond their visual shape. Each component type should understand its own domain semantics.

Database component	Understands tables, fields, relationships, and constraints
Service component	Understands APIs, ports, dependencies, and protocols
Class component	Understands methods, properties, visibility, and relationships
Sequence participant	Understands message ordering, lifelines, and activation bars

6.8 Smart Connection System

Connections between components should behave intelligently and maintain their semantic meaning.

Expected capabilities:

- Create typed connections (foreign key, dependency, API call, inheritance, etc.)
- Maintain connections when components are repositioned
- Label connections with names, multiplicity, and direction
- Show directional arrows and dependency types visually
- Prevent semantically invalid connections per workspace type

6.9 Collaboration System

Engineering diagrams are collaborative documents that multiple people work on simultaneously.

Real-time capabilities:

- Multiple active editors on the same diagram
- Real-time diagram state synchronisation
- Live user presence indicators
- Cursor tracking per active collaborator
- Simultaneous editing without conflicts

Concurrent Editing Requirement

When multiple users modify the same diagram simultaneously:

- Changes must remain consistent across all clients
- Work must not be unexpectedly lost
- Diagram state must remain reliable at all times

6.10 Version History System

Software architecture evolves continuously. FrameLabs must preserve that evolution.

Expected capabilities:

- Automatic tracking of all diagram changes
- View and browse previous diagram versions
- Restore any older diagram state
- Understand what changed between versions
- Attribute changes to specific collaborators

6.11 Organisation System

Users should manage large collections of diagrams across projects and teams.

Expected capabilities:

- Workspaces for isolating team or project diagrams
- Projects for grouping related diagrams
- Tags and categories for classification
- Search across diagrams, components, and metadata

6.12 Export & Sharing System

Engineering diagrams should integrate smoothly with existing documentation and development workflows.

Export formats:

- Images (PNG, SVG)
- PDFs for documentation packages
- Documentation-friendly formats (Markdown embed, etc.)

Sharing options:

- Public read-only links
- Team-scoped access controls
- Permission-controlled diagram sharing

7. Product Scale Requirements

FrameLabs should be designed beyond small personal diagrams and must support growing engineering organisations.

User Scale

Registered users	100,000+
Monthly active users	10,000+
Concurrent users	500+

Workspace Scale

Engineering workspaces	50,000+
Diagrams per organisation	Thousands

Diagram Scale

Total diagrams	500,000+
Objects in large diagrams	50,000+
Total object relationships	Millions

Collaboration Scale

Active collaborative diagrams	100+
Users editing one workspace	50+
Diagram operations per minute	Thousands

Version Scale

Diagram revisions	Millions
-------------------	----------

Architecture history retention	Long-term (no expiry)
---------------------------------------	-----------------------

8. Product Constraints

In Scope

- Web application (browser-based)
- Dedicated diagram workspaces per type
- Visual drag-and-drop editing
- Syntax-based diagram creation
- Real-time collaboration
- Version history
- Export and sharing

Out of Scope

Not expected in v1.0

General-purpose design or whiteboard tool

Image editing features

Presentation builder

Native mobile application

Marketplace or plugin system

9. Technical Expectations

Technology decisions belong to the engineering team. The platform should demonstrate the following non-functional qualities:

Reliability	Technical diagrams remain safe, consistent, and never silently corrupted
Performance	Large diagrams (50k+ objects) remain responsive; collaboration is low-latency
Maintainability	New diagram workspace types can be introduced without architectural rewrites
Scalability	Platform supports growing teams and millions of versioned diagram revisions

10. Success Metrics

Product Success

- Engineers can design real software systems, not toy diagrams
- Each diagram type feels purpose-built for its domain
- Visual and syntax workflows operate seamlessly together
- Teams can collaborate on diagrams in real time
- Architecture history is preserved and browsable

Engineering Success

- Clean, maintainable architecture decisions
- Stable, repeatable deployments
- Well-structured source repository
- Comprehensive documentation

11. Final Deliverables

Product Deliverables

- Deployed and accessible engineering diagramming platform
- Multiple specialised diagram workspaces
- Real-time collaboration experience
- Export and sharing functionality

Engineering Deliverables

- Source code repository
- System architecture documentation
- API documentation
- System design documentation
- Setup and deployment instructions

Final Objective

Build a professional engineering diagram workspace where developers can design, document, and evolve software systems.

The challenge is not building a drawing canvas.

The challenge is creating a structured environment that understands engineering concepts and transforms them into collaborative visual systems.